

TEMA 7

FICHEROS



PROGRAMACIÓN

JAKELIN SAYEL

"El mundo cambia con tu
ejemplo no con tu
opinión".

Paulo Coelho



ÍNDICE

1. **Introducción**
2. **Stream**
3. **Ficheros. Acceso con Flujo de Datos.**
4. **Ficheros.Flujo Caracteres con BufferedReader**
5. **La Clase File:**
 - **Rutas Absolutas y Relativas**
 - **Métodos de la Clase File**
 - **Comprobaciones de Estado**
 - **Propiedades de Ficheros**
 - **Gestión de Ficheros**
 - **Listado de Archivos**
6. **Acceso a Ficheros con Flujo de Bytes**
7. **Serialización**



1. INTRODUCCIÓN

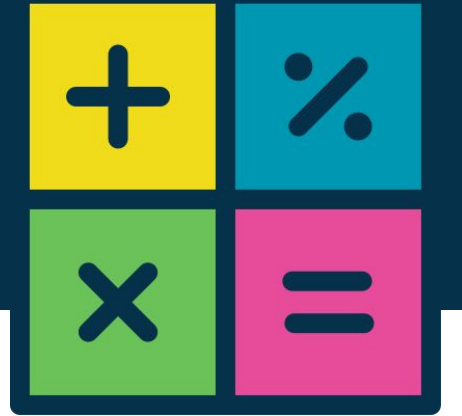


¿Por qué el uso de ficheros?

- Toda la información que guardamos en un programa de java, lo hacemos en variables, objetos, arrays... y se guarda en la memoria RAM del ordenador. Es decir, que como la memoria Ram es volátil, cuando termina la ejecución del programa esa información se pierde.
- Para solucionar esto tenemos en acceso a ficheros externos, es decir, tener la posibilidad con un programa de java tanto leer ficheros externos como escribir ficheros externos, así podremos almacenar información de forma permanente.
- Podemos almacenar información gracias a los STREAMS (flujos) de datos, por ejemplo en pendrives, discos duros...
- Con lo que podremos recuperar la información en la siguiente ejecución del programa, o incluso desde otros programas de java o en otros lenguajes de programación



1. INTRODUCCIÓN



Operaciones con Ficheros

→ Un archivo o **fichero informático** es un conjunto de bytes que son almacenados en un dispositivo de almacenamiento permanente.

- Creación
- Consulta o Recuperación
- Mantenimiento o actualización
- Borrado o destrucción.

Organización de Ficheros

→ El acceso a los datos de un fichero está ligado a la organización de éste.

→ Existen dos tipos de accesos:

- ◆ **Acceso Secuencial:** para acceder al registro n hay que recorrer los $n-1$ anteriores.
- ◆ **Acceso Directo:** Se puede acceder directamente a un registro a partir de una clave de acceso.



1. INTRODUCCIÓN

E/S en java

- Los programas de Java realizan la **E/S a través de flujos (stream)**.
- Por tanto, se pueden aplicar **las mismas clases y métodos E/S a cualquier** tipo de **dispositivo**.
- **Java implementa los flujos en las clases incluidas en el paquete java.io**
- El acceso a Ficheros se hará mediante operaciones de Entrada/Salida.
- El sistema de E/S de Java es extenso, con numerosas clases, interfaces y métodos.
- Java define dos sistemas completos de E/S:
 - ◆ Para bytes
 - ◆ Para caracteres

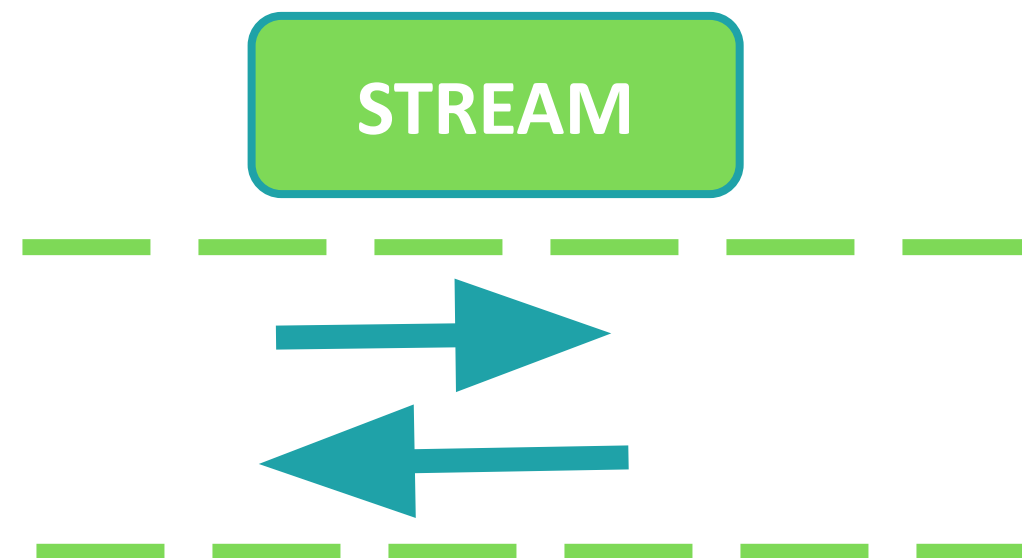


2. STREAMS

¿Qué son los Streams de Datos?

- Son una especie de puentes que se crean entre nuestro programa java y el dispositivo de almacenamiento

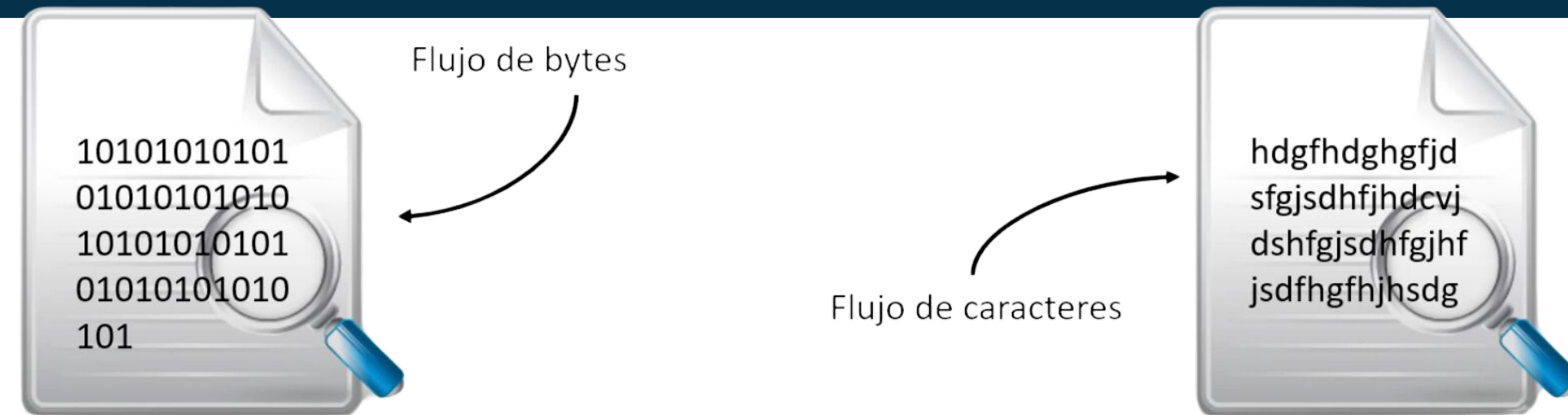
```
package excepciones_I;  
  
import javax.swing.JOptionPane;  
  
public class Aleatorios {  
  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
  
        int elementos=Integer.parseInt(JOptionPane.sh  
  
        int num_aleat[]=new int[elementos];  
  
        for(int i=0;i<num_aleat.length;i++){  
            num_aleat[i]=(int)(Math.random()*100);  
        }  
  
        for(int elem: num_aleat){  
            System.out.println(elem);  
        }  
    }  
}
```



- Es por donde fluye la información, por eso también, se les llama **flujos de datos**.
- Fluye en ambos sentidos del programa al dispositivo es escritura.
- Desde el dispositivo al programa es una lectura

2. STREAMS

Dos formas de abordar los STREAMS: Flujos de Bytes y Caracteres



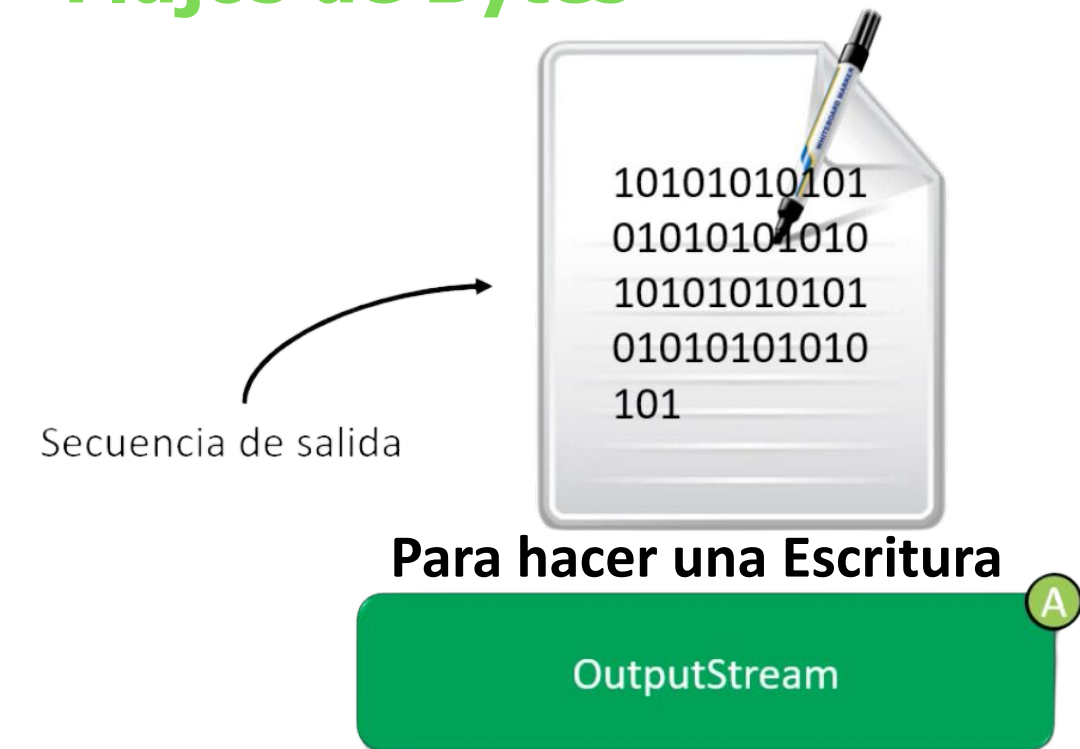
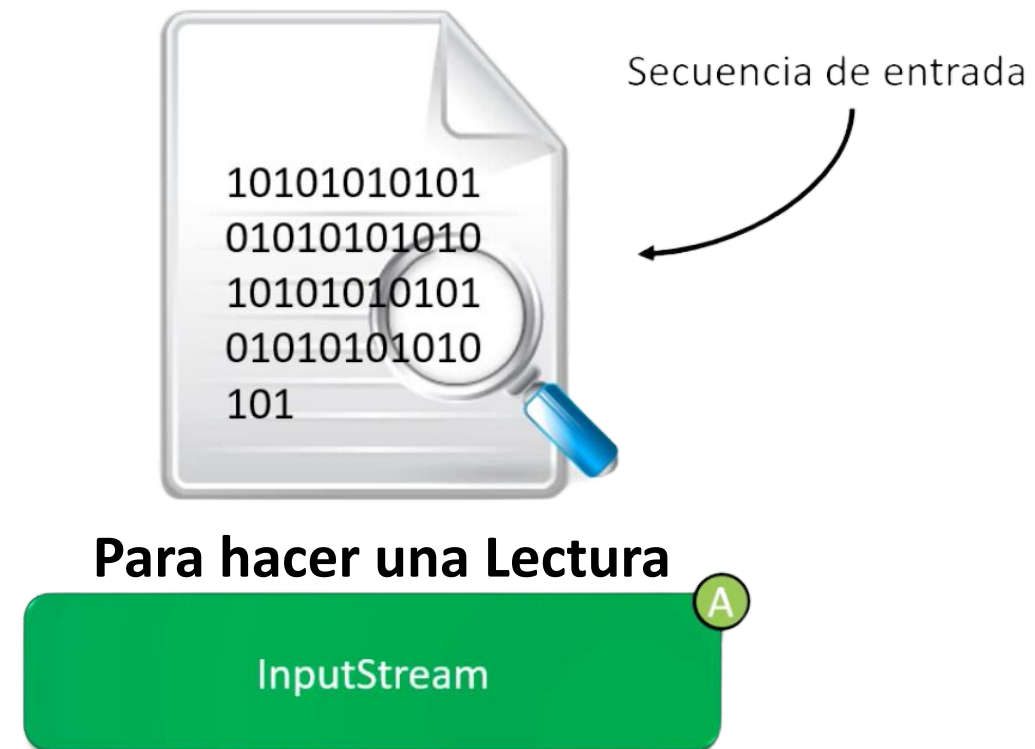
→ En Java hay 2 formas de recibir o enviar datos:

- ◆ **Flujo de caracteres** : permiten procesar la E/S de caracteres. Usan UNICODE. Ficheros de texto.
 - ◆ **Flujo bytes**: permiten procesar la E/S de bytes. Uso por ejemplo para leer y escribir datos binarios, Útiles para trabajar con archivos, enviar paquetes de datos a través de una red,...
- Usaremos una u otra en función de lo que queramos hacer.
- Por ejemplo, si queremos leer o escribir un fichero de texto (word o web o lo que sea) usaremos un flujo de caracteres.
- Si queremos guardar un fichero donde guardamos un objeto, entonces usaremos flujos de bytes

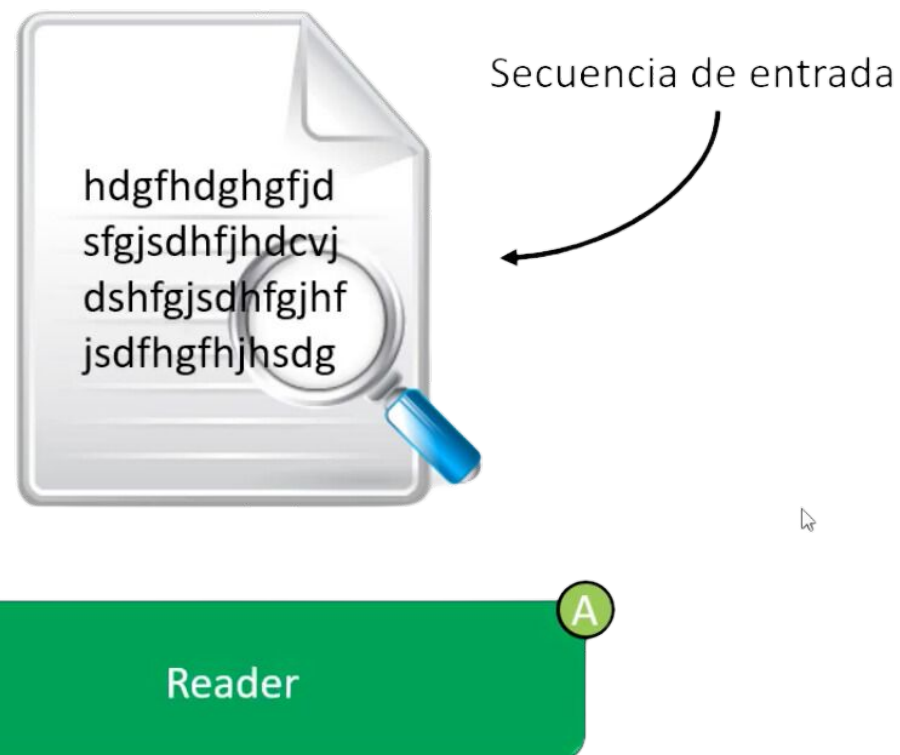
2. STREAMS

→ Dependiendo del Stream que vamos a manejar (bytes o caracteres), usaremos unas clases u otras.

Flujos de Bytes

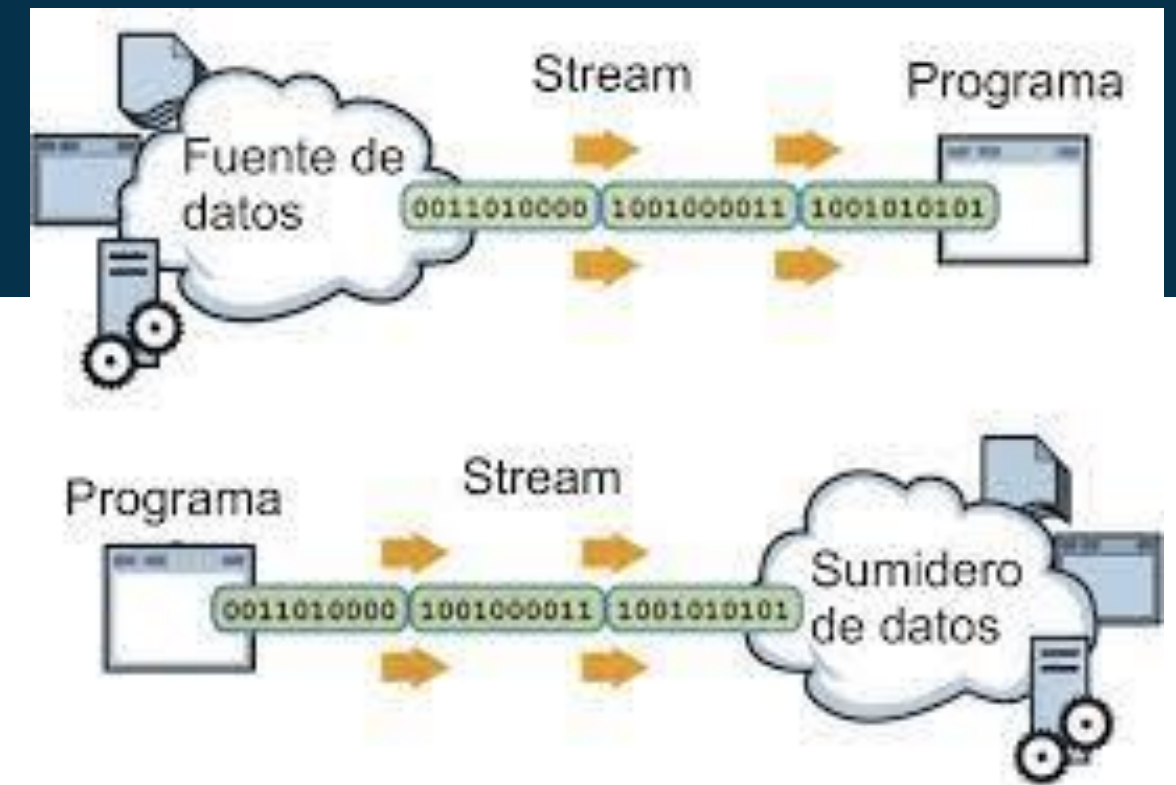


Flujos de Caracteres



→ Las clases son clases Abstractas con una cadena de herencia muy larga, nosotros nos centraremos en las principales,

2. STREAMS



Clases de Flujos de Caracteres:

- Se definen con dos jerarquías de clases:
 - ◆ **Reader** (características flujos entrada)
 - ◆ **Writer** (comportamiento flujos de salida)



Secuencia de entrada



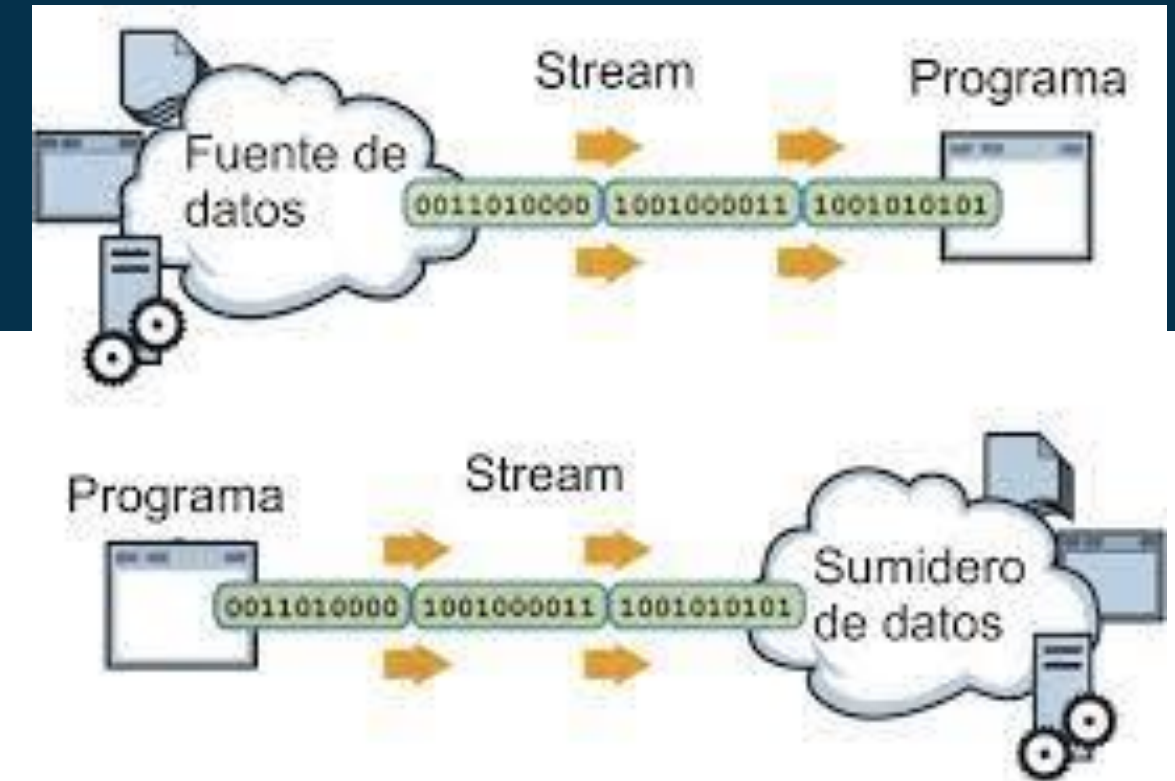
Secuencia de salida

Reader

Writer

Paquete Java.IO

2. STREAMS

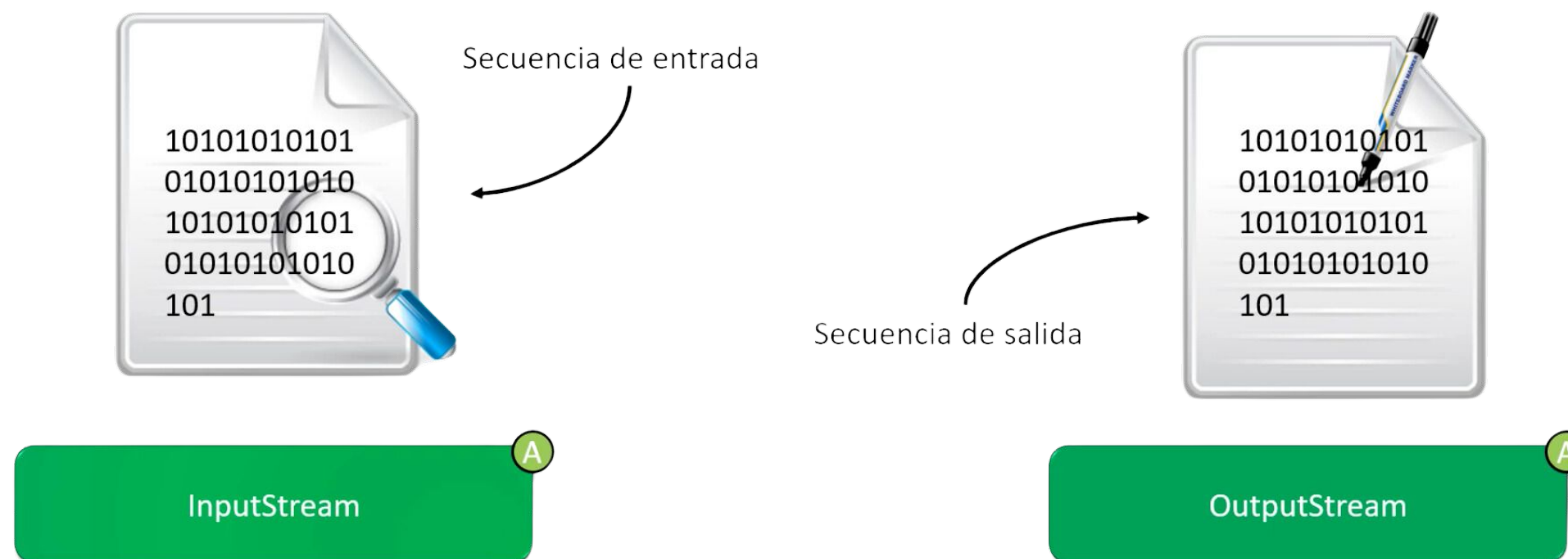


Clases de Flujos de Bytes

→ Se definen con dos jerarquías de clases:

- ◆ **InputStream** (características flujos entrada)
- ◆ **OutputStream** (comportamiento flujos de salida)

A partir de estas dos clases se crean otras subclases concretas de distinta funcionalidad y que se encargan de los detalles de leer y escribir en distintos dispositivos (ejem. Archivos en disco)



Paquete Java.IO

3. FICHEROS. ACCESO CON FLUJO DE CARACTERES



- ◆ Se definen con dos jerarquías de clases:
 - **Reader** (características flujos entrada)
 - **Writer** (comportamiento flujos de salida)
- ◆ **Ventaja:** operan directamente con caracteres Unicode. Por tanto, para almacenar un texto Unicode es la mejor opción.
- ◆ Por lo general, para realizar E/S de archivos basada en caracteres , se usan las clases: `FileWriter` y `FileReader`
- ◆ Abrimos el fichero de lectura y leeremos la información de forma secuencial.

CONSTRUCTORES DE LA CLASE `FileReader`:

```
FileReader nombre = new FileReader("fichero.extension");  
(Controlar la excepción FileNotFoundException)
```

3. FICHEROS. ACCESO CON FLUJO DE CARACTERES

READER

- Es conveniente ir a las clases en la API de java. Por ejemplo, para la clase **Reader**
- Lo que hace **InputStreamReader** es transformar el binario a al correspondiente carácter Unicode. Una subclase es **FileReader**.

Class Reader

java.lang.Object
java.io.Reader

All Implemented Interfaces:

Closeable, AutoCloseable, Readable

Direct Known Subclasses:

BufferedReader, CharArrayReader, FilterReader, **InputStreamReader**, PipedReader, StringReader

```
public abstract class Reader
extends Object
implements Readable, Closeable
```

Pinchar sobre ella y ver la descripción es importante

Class InputStreamReader

java.lang.Object
java.io.Reader
java.io.InputStreamReader

All Implemented Interfaces:

Closeable, AutoCloseable, Readable

Direct Known Subclasses:

FileReader

Constructors

Constructor and Description

FileReader(File file)
Creates a new FileReader, given the File to read from.

FileReader(FileDescriptor fd)
Creates a new FileReader, given the FileDescriptor to read from.

FileReader(String fileName)
Creates a new FileReader, given the name of the file to read from.

3. FICHEROS. ACCESO CON FLUJO DE CARACTERES



Muy importante!!

- Todas estas clases pertenecen al paquete **java.io** (tendremos que importarlo).
- Además, si por ejemplo nos fijamos en el primer constructor que vamos a usar (FileReader):

```
FileReader  
  
public FileReader(File file)  
    throws FileNotFoundException  
  
Creates a new FileReader, given the File to read from.
```

Lanza una Excepción!!!

- También, es importante cerrar el Stream una vez , hayamos terminado de usarlo:
nombreStream.close();

3. FICHEROS. ACCESO CON FLUJO DE CARACTERES

WRITER

- Es conveniente ir a las clases en la API de java. Por ejemplo, para la clase **Writer**
- Lo que hace **OutputStreamWriter** Si pulsamos sobre ella nos lleva a la clase **FileWriter**, que es la que necesitamos.
- Si pulsamos sobre ella (FileWriter), la descripción nos dice que escribe archivos de caracteres, también, vemos que tiene sobrecarga de constructores.

java.io

Class Writer

java.lang.Object
java.io.Writer

All Implemented Interfaces:
Closeable, Flushable, Appendable, AutoCloseable

Direct Known Subclasses:
BufferedWriter, CharArrayWriter, FilterWriter, **OutputStreamWriter**, PipedWriter, PrintWriter, StringWriter

Pinchar sobre ella y ver la descripción es importante

Class OutputStreamWriter

java.lang.Object
java.io.Writer
java.io.OutputStreamWriter

All Implemented Interfaces:
Closeable, Flushable, Appendable, AutoCloseable

Direct Known Subclasses:
FileWriter

Constructors

Constructor and Description
FileWriter(File file) Constructs a FileWriter object given a File object.
FileWriter(File file, boolean append) Constructs a FileWriter object given a File object.
FileWriter(FileDescriptor fd) Constructs a FileWriter object associated with a file descriptor.
FileWriter(String fileName) Constructs a FileWriter object given a file name.
FileWriter(String fileName, boolean append) Constructs a FileWriter object given a file name with a boolean indi

3. FICHEROS. ACCESO CON FLUJO DE CARACTERES

- Abrimos el fichero de escritura, y guardaremos la información carácter a carácter.

CONSTRUCTORES :

FileWriter nombre = new FileWriter("fichero.extension"); //Crear un fichero nuevo y escribir en él

FileWriter nombre = new FileWriter("fichero.extension", boolean append);

a= true añade datos al fichero que ya existe.

a= false sobrescribe el fichero

// Agregar texto al texto que ya tenemos, es decir, si es true agregamos algo al fichero y si es false no queremos agregar nada. **Aquí escribimos algo en un fichero existente.**

MÉTODOS

void write(caracter): escribe el carácter en el fichero.

PARA CERRAR EL CANAL

void close(), cierra el fichero.

3. FICHEROS. ACCESO CON FLUJO DE CARACTERES



Muy importante!!

- Todas estas clases pertenecen al paquete **java.io** (tendremos que importarlo).
- Además, si por ejemplo nos fijamos en el primer constructor que vamos a usar (FileWriter):

```
public FileWriter(File file)  
    throws IOException
```



Lanza una Excepción!!!

ALgunos MÉTODOS

- **int read()**: devuelve en ASCII el byte leído. Nos devuelve **-1** si no tiene más información el fichero. En algún caso puede que falle el -1 y debemos preguntar por 65535.
- **int skip(long)**: salta n caracteres.
- **String readLine()**: devuelve toda una línea del fichero.
- **close()** : cierra el stream



4. F.F. DE CARACTERES: FILEREADER con BufferedReader



Ficheros Flujo CARACTERES - BufferedReader

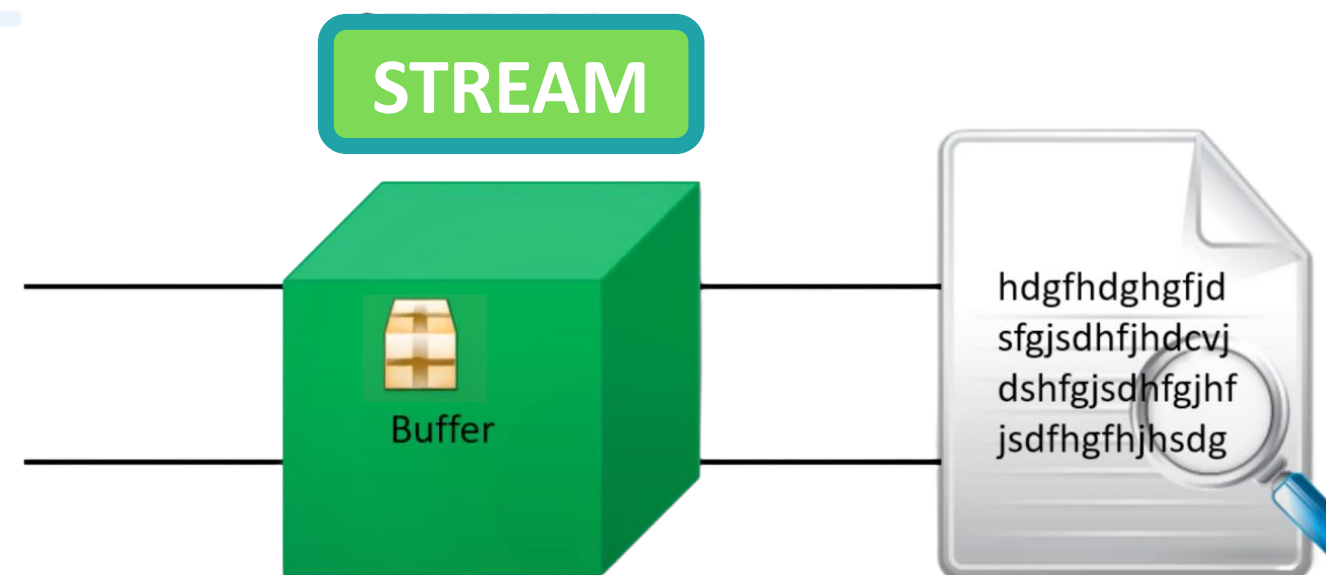
- Seguimos viendo cómo acceder a un fichero externo para leer la información de su interior usando Stream de datos (flujo de datos) pero ahora mediante BUFFERS.
- Con FileReader hemos visto que podemos acceder a los ficheros de texto pero solo si tiene poca información, cuando hay mucho texto no lo debemos utilizar porque le programa tardaría muchísimo en acceder a toda la información y en ese proceso, podría ocurrir alguna interrupción a la información.
- ¿Qué son lo Buffers?: En realidad usamos lo Buffers en nuestro día a día, por ejemplo si vemos un vídeo online y la conexión se interrumpe, el video se pausa y podemos ver Buffering... (el buffer se está rellenado).
- Un **buffer** sería una memoria que se sitúa en un punto intermedio entre nuestro programa y el fichero al que queremos acceder.



4. F.F. DE CARACTERES: FILEREADER con BufferedReader

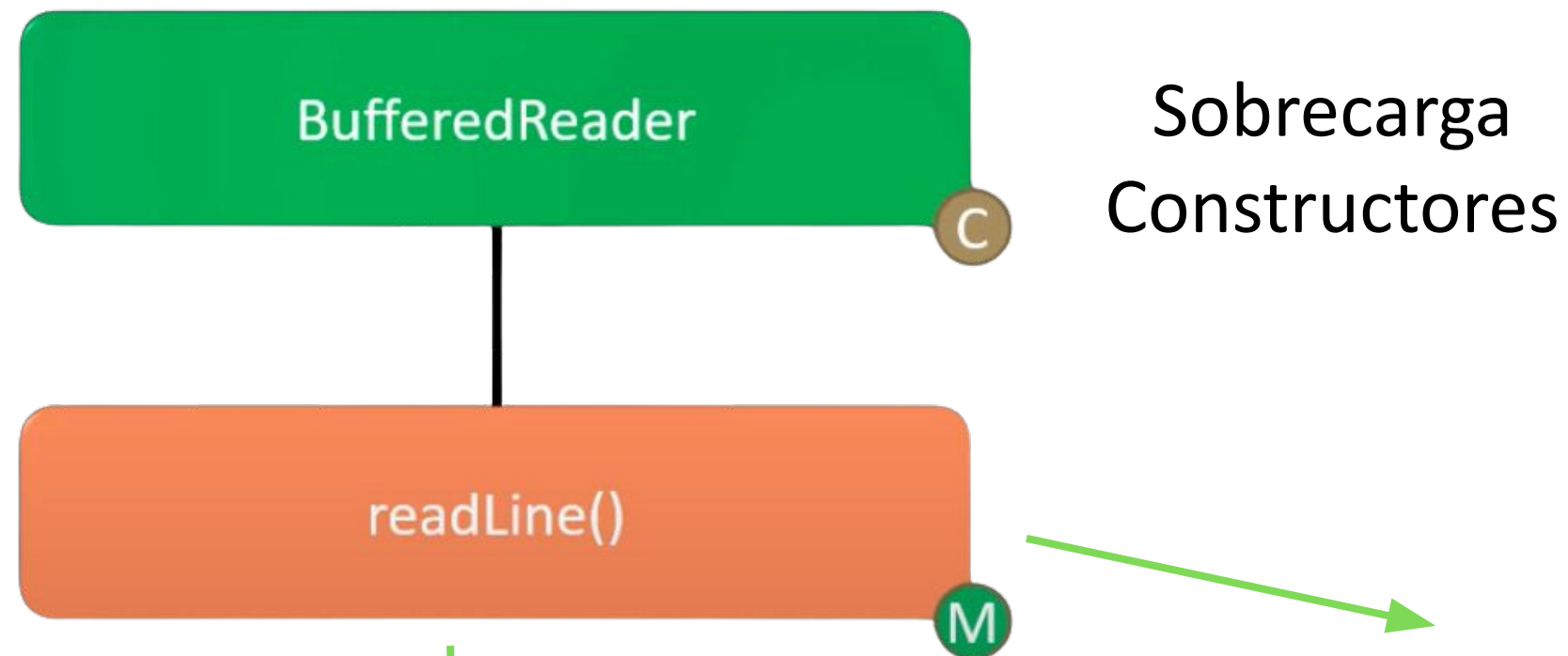
- Esta memoria se rellena con información antes de que podamos acceder a ella, es decir, cuando solicitamos información de un fichero lo primero es que se rellena el buffer con esa información
- Una vez que ha sido rellenada es cuando comienza el acceso a la información.
- Así pues, el acceso no lo hacemos sobre el propio fichero externo como antes, sino que haces el acceso al Buffer que es quien tiene la información
- Si por ejemplo, hay un problema con el disco duro donde tenemos el fichero, la lectura no se interrumpe.
- Este buffer se rellena en nuestra Ram, si hubiese un problema en la Ram ahí sí, se interrumpe.

```
}  
}  
  
class Leer_Fichero {  
    public void lee() {  
  
        try {  
            entrada = new FileReader("C:/Users/Juan/Desktop/ejemplo.txt");  
  
            int c = 0;  
  
            while (c != -1) {  
  
                c = entrada.read();  
  
                char letra = (char) c;  
  
                System.out.print(letra);  
  
            }  
  
            // entrada.close();  
  
        } catch (IOException e) {  
            // TODO Auto-generated catch block  
            System.out.println("No se ha encontrado el archivo");  
        } finally {  
  
            try {  
                entrada.close();  
            } catch (IOException e) {  
                // TODO Auto-generated catch block  
                e.printStackTrace();  
            }  
        }  
    }  
}
```



4. F.F. DE CARACTERES: FILEREADER con BufferedReader

- Resumiendo, se utiliza cuando queremos acceder a mucha información e incluso cuando queremos acceder a información remota en otra máquina.



```
readLine
```

```
public String readLine()
    throws IOException
```

Reads a line of text. A line is considered to be terminated by any one of a line feed ('\n'), a carriage return ('\r'), or a carriage return followed immediately by a linefeed.

Returns:
A String containing the contents of the line, not including any line-termination characters, or null if the end of the stream has been reached

Throws:
IOException - If an I/O error occurs

See Also:
Files.readAllLines(java.nio.file.Path, java.nio.charset.Charset)

Constructors

Constructor and Description

BufferedReader(Reader in)

Creates a buffering character-input stream that uses a default-sized input buffer.

BufferedReader(Reader in, int sz)

Creates a buffering character-input stream that uses an input buffer of the specified size.

Aquí especificamos el
fichero Externo

Aquí podemos decir
el tamaño de buffer

Lee una Línea de texto hasta
que encuentra un **salto de
línea** o un **retorno de carro**.
Cuando llega al final del texto
devuelve **NULL**. Además, lanza
una **Excepción de tipo IO**

4. F.F. DE CARACTERES: FILEREADER con BufferedReader

- Si cada registro está almacenado en una fila del fichero podemos leer todos los datos de una línea y no carácter a carácter.
- Debemos utilizar **BufferedReader**
- Recordar uso de BufferedReader

```
import java.io.*;
```

BufferedReader <nombre_objeto> =new BufferedReader(objeto clase FileReader);

<variable>=<nombre_objeto>.readLine();

Preguntaremos por distinto de null, en la lectura del fichero..



4. F.F. DE CARACTERES: FILEREADER con BufferedReader

Abrimos el fichero, y guardaremos la información carácter a carácter.

CONSTRUCTORES :

BufferedReader(objeto Reader);

BufferedReader(objeto Reader, int tamaño);

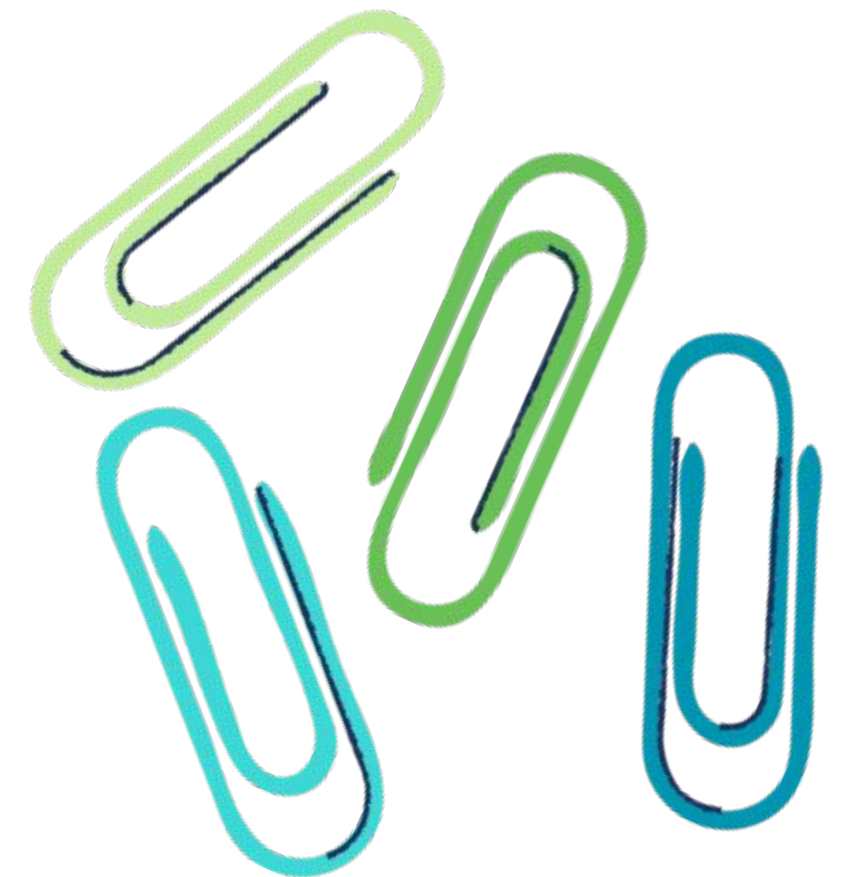
MÉTODOS

Int read(): leer un carácter del fichero.

String readLine(): leer una línea de texto.

PARA CERRAR EL CANAL

void close(), cierra el fichero.



5. LA CLASE FILE:

- La pieza más básica para poder operar con archivos, independientemente de su tipo, en un programa Java es la **clase File**.
- Esta clase pertenece al package **java.io**. Por lo tanto será necesario importarla antes de poder usarla: **import java.io.File;**
- Esta clase permite manipular cualquier aspecto vinculado al sistema de ficheros. Su nombre ("archivo", en inglés) es un poco engañoso, ya que no se refiere exactamente a un archivo.

La clase File representa una ruta dentro del sistema de archivos.

- Sirve para realizar operaciones tanto sobre rutas al sistema de archivos **que ya existan como no existentes**.
- Se puede usar tanto para **manipular archivos, carpetas y directorios**.
- Como cualquier otra clase hay que **instanciarla** para que sea posible invocar sus métodos. El constructor de File recibe como **argumento una cadena de texto correspondiente a la ruta** sobre la que se quieren llevar a cabo las operaciones.

5. LA CLASE FILE:

```
File f = new File (String ruta);
```

- Una ruta es la forma general de un nombre de archivo o carpeta, por lo que identifica únicamente su localización en el sistema de archivos.
- Cada uno de los elementos de la ruta pueden existir realmente o no, pero esto no impide en modo poder inicializar FILE.
- Un aspecto importante a tener presente al inicializar File es tener siempre presente que el formato de la cadena de texto que conforma **la ruta puede ser diferente según el sistema operativo** sobre el que se ejecuta la aplicación.
- **Por ejemplo**, el sistema operativo Windows inicia las rutas por un nombre de unidad (C :, D :, etc.), mientras que los sistemas operativos basados en Unix comienzan directamente con una barra ("/"). Además, los diferentes sistemas operativos usan diferentes separadores dentro de las rutas. Por ejemplo, los sistemas Unix usan la barra ("/") mientras que el Windows la inversa ("\\").

5. LA CLASE FILE:

- Ejemplo de ruta Unix: /usr/bin
- Ejemplo de ruta Windows: C:\Windows\System32
- De todos modos Java nos permite utilizar la barra de Unix ("/") para representar rutas en sistemas Windows. Por lo tanto, es posible utilizar siempre este tipo de barra independientemente del sistema, por simplicidad.
- Es importante entender que **un objeto representa una única ruta del sistema de ficheros**
- Para operar con diferentes rutas a la vez habrá que crear y manipular varios objetos.
- **Por ejemplo:** Código se instancian tres objetos File diferentes.
 - File carpetaFotos = new File("C:/Fotos");
 - File unaFoto = new File("C:/Fotos/Foto1.png");
 - File otraFoto = new File("C:/Fotos/Foto2.png");



5. LA CLASE FILE:

Rutas Absolutas y Relativas

- En los ejemplos empleados hasta el momento para crear objetos del tipo File se han usado **rutas absolutas**, ya que es la manera de dejar más claro a qué elemento dentro del sistema de archivos, ya sea archivo o carpeta, se está haciendo referencia.
- Una ruta absoluta es aquella que se refiere a un elemento a partir del raíz del sistema de ficheros. Por ejemplo "C:/Fotos/Foto1.png"
- Al instanciar objetos de tipo File usando una ruta absoluta siempre hay que usar la representación correcta según el sistema en que se ejecuta el programa.
- Si bien el uso de rutas absolutas resulta útil para indicar con toda claridad qué elemento dentro del sistema de archivos se está manipulando, hay casos que su uso conlleva ciertas complicaciones.
- Solución: **Rutas Relativas**

5. LA CLASE FILE:

Rutas Absolutas y Relativas

- Una **ruta relativa** es aquella que **no incluye el raíz** y por ello se considera que **parte desde el directorio de trabajo de la aplicación**. Esta carpeta puede ser diferente cada vez que se ejecuta el programa.
- Cuando un programa se ejecuta por defecto se le asigna una carpeta de trabajo. Esta carpeta suele ser la carpeta desde donde se lanza el programa. En el caso de un programa en Java ejecutado a través de un IDE (como Eclipse), la carpeta de trabajo suele ser la misma carpeta donde se ha elegido guardar los archivos del proyecto (Workspace-proyecto).
- El formato de una ruta relativa es similar a una ruta absoluta, pero **nunca se indica la raíz** del sistema de ficheros.
- Directamente se empieza por el primer elemento escogido dentro de la ruta. Por ejemplo:
 - Viajes
 - Unidad7\apartado1
 - Unidad7\apartado1\Actividades.txt

5. LA CLASE FILE:

Rutas Absolutas y Relativas

- Una **ruta relativa siempre incluye el directorio de trabajo de la aplicación como parte inicial** a pesar de no haberse escrito.
- El rasgo distintivo es que el directorio de trabajo puede variar. Por ejemplo, el elemento al que se refiere el siguiente objeto File varía según el directorio de trabajo.
 - `File f = new File ("Unidad7/apartado1/Actividades.txt");`
- Este mecanismo permite facilitar la portabilidad del software entre distintos ordenadores y sistemas operativos, ya que solo es necesario que los archivos y carpetas permanezcan en la misma ruta relativa al directorio de trabajo. Veámoslo con un ejemplo:
 - `File f = new File ("Activdades.txt");`
- Dada esta ruta relativa, basta garantizar que el fichero "Activdades.txt" esté siempre en el mismo directorio de trabajo de la aplicación, cualquiera que sea éste e independientemente del sistema operativo utilizado (en un ordenador puede ser "C:\Programas" y en otro "/Java"). **En cualquiera de todos estos casos, la ruta siempre será correcta.**

5. LA CLASE FILE:

Rutas Absolutas y Relativas

- Como sabemos java es un lenguaje multiplataforma, por eso el tema de las rutas es tan importante porque si queremos trasladar nuestro programa a otro ordenador debería funcionar y como hemos visto según el tipo de SSOO, incluso aunque sea Windows ambos puede dar problemas a la hora de las barras.
- Por eso en la API de File podemos encontrar unos campos de clase:

Fields	
Modifier and Type	Field and Description
static String	pathSeparator The system-dependent path-separator character, represented as a string for convenience.
static char	pathSeparatorChar The system-dependent path-separator character.
static String	separator The system-dependent default name-separator character, represented as a string for convenience.
static char	separatorChar The system-dependent default name-separator character.

Lo que hace es especificar el separador por defecto del SSOO en el que nos encontramos **SIEMPRE QUE EL FICHERO SE ENCUENTRE EN LA MISMA RUTA**

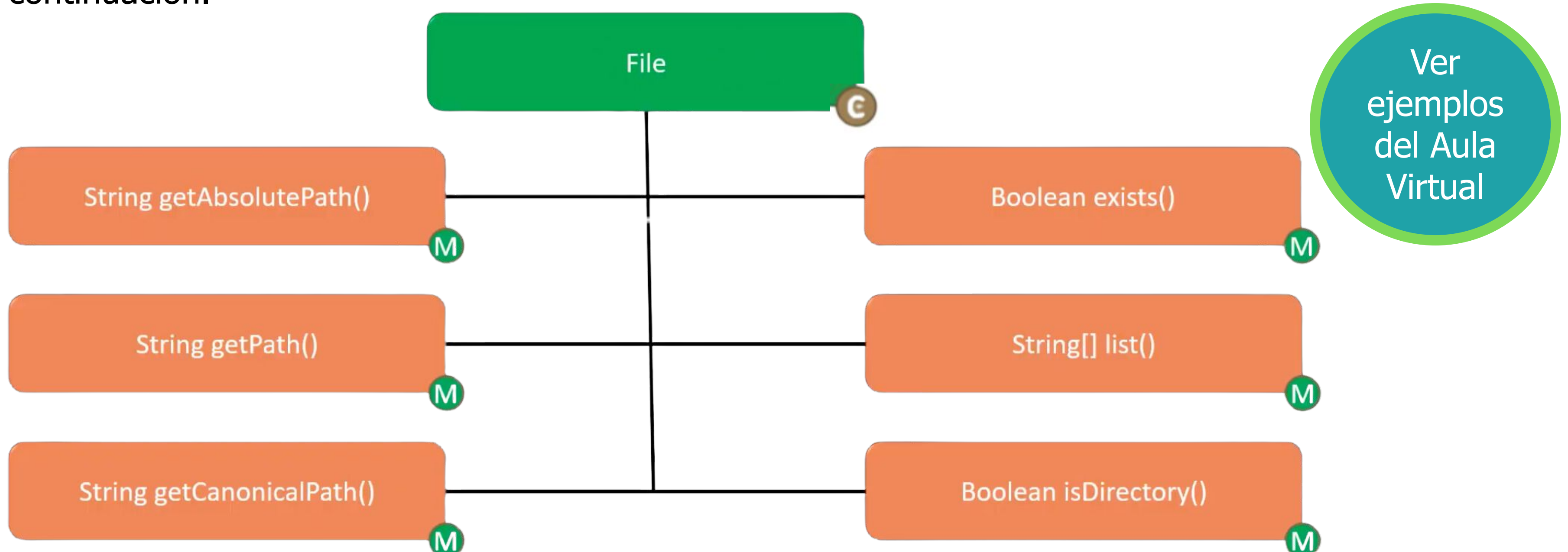
"C"+ File.**separator**+ "Users"+File.**separator**+"jakelin"+File.**separator**+ "Desktop"+File.**separator**+ "Documentos_Ficheros"+ File.**separator**+"Libros"

iiiiiii Nosotros usaremos RUTAS RELATIVAS en el EXAMEN!!!!!!!

5. LA CLASE FILE:

Métodos de la Clase File

- File ofrece varios métodos para poder manipular el sistema de archivos u obtener información a partir de su ruta. Algunos de los más significativos para entender las funcionalidades se muestran a continuación.



5. LA CLASE FILE:

Métodos de la Clase File: Obtención de la ruta

Una vez se ha instanciado un objeto de tipo File, puede ser necesario recuperar la información empleada durante su inicialización y conocer en formato texto a qué ruta se está refiriendo, o al menos parte de ella. **SIEMPRE CONSULTAR LA API**

- String **getParent()** devuelve la ruta de la carpeta del elemento referido por esta ruta. Básicamente la cadena de texto resultante es idéntica a la ruta original, eliminando el último elemento. Si la ruta tratada se refiere a la carpeta raíz de un sistema de archivos ("C:\\", "/", etc.), este método devuelve null. En el caso de tratarse de una ruta relativa, este método no incluye la parte de la carpeta de trabajo.
- String **getName()** devuelve el nombre del elemento que representa la ruta, ya sea una carpeta o un archivo. Es el caso inverso del método getParent(), ya que el texto resultante es solo el último elemento.
- String **getAbsolutePath()** devuelve la ruta absoluta. Si el objeto File se inicializó usando una ruta relativa, el resultado incluye también la carpeta de trabajo.

5. LA CLASE FILE:

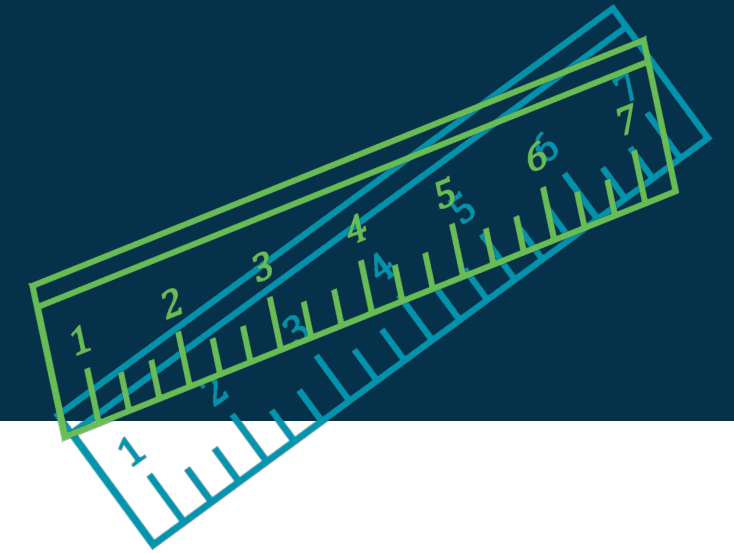
Métodos de la Clase File: Comprobaciones de estado

Dada la ruta empleada para inicializar una variable de tipo File, esta puede que realmente exista dentro del sistema de ficheros o no, ya sea en forma de archivo o carpeta. La clase File ofrece un conjunto de métodos que permiten hacer comprobaciones sobre su estado y saber si es así.

- boolean **exists()** comprueba si la ruta existe dentro del sistema de ficheros. Devolverá true si existe y false en caso contrario.
- boolean **isFile()** comprueba el sistema de ficheros en busca de la ruta y devuelve true si existe y es un fichero. Devolverá false si no existe, o si existe pero no es un fichero.
- boolean **isDirectory()** funciona como el anterior pero comprueba si es una carpeta.



5. LA CLASE FILE:



Métodos de la Clase File: Propiedades de Ficheros

- long **length()** devuelve el tamaño de un archivo en bytes. Este método solo puede ser llamado sobre una ruta que represente un archivo, de lo contrario no se puede garantizar que el resultado sea válido.
- long **lastModified()** devuelve la última fecha de edición del elemento representado por esta ruta. El resultado se codifica en un único número entero cuyo valor es el número de milisegundos que han pasado desde el 1 de junio de 1970.

```
import java.io.*;
import java.util.*;
//ejemplo propiedades de Ficheros
public class AccesoRuta_04 {
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        File docu= new File
("/Users/jakelinsayel/Desktop/Documentos_Ficheros/Fotografias");
        long milisegundos=docu.lastModified();
        Date fecha=new Date(milisegundos);

        System.out.println(docu.getAbsolutePath());
        System.out.println("Última modificación(ms): " + milisegundos);
        System.out.println("Última modificación(fecha): " + fecha);
        System.out.println("Tamaño archivo: " + docu.length());
    }
}
```

5. LA CLASE FILE:

Métodos de la Clase File: Gestión de ficheros

- boolean **mkdir()** permite crear la carpeta indicada en la ruta. La ruta debe indicar el nombre de una carpeta que no existe en el momento de invocar el método. Por ejemplo, dado un objeto File instanciado con la ruta "C:/Fotos/Albania" que no existe, el método mkdir() creará la carpeta "Albania" dentro de "C:/Fotos". Devuelve true si se ha creado correctamente, en caso contrario devuelve false (por ejemplo si la ruta es incorrecta, la carpeta ya existe o el usuario no tiene permisos de escritura).



```
public class CrearDirectorios_00 {  
    public static void main(String[] args) {
```

```
        File directorio = new  
        File("/Users/jakelinsl/Desktop/Documentos_Ficheros/Libros/NuevoDirectorio");
```

```
        //NuevoDirectorio es el directorio que vamos a crear y lo ponemos al final de la  
        ruta
```

```
        directorio.mkdir(); // crea el directorio
```

```
    } }
```

5. LA CLASE FILE:

DELETE



Métodos de la Clase File: Gestión de ficheros

- boolean **delete()** borra el archivo o carpeta indicada en la ruta. La ruta debe indicar el nombre de un archivo o carpeta que sí existe en el momento de invocar el método. Se podrá borrar una carpeta solo si está vacía (no contiene ni carpetas ni archivos). Devuelve true o false según si la operación se ha podido llevar a cabo.
- boolean **renameTo(File destino)** el nombre de este método es algo engañoso ("renombrar", en inglés), ya que su función real no es simplemente cambiar el nombre de un archivo o carpeta, sino cambiar la ubicación completa. El método se invoca el objeto File con la ruta origen (donde se encuentra el archivo o carpeta), y se le da como argumento otro objeto File con la ruta destino. Devuelve true o false según si la operación se ha podido llevar a cabo correctamente o no (la ruta origen y destino son correctos, no existe ya un archivo con este nombre en el destino, etc.). Nótese que, en el caso de carpetas, es posible moverlas aunque contengan archivos.

5. LA CLASE FILE:



Métodos de la Clase File: Listado de archivos

→ **File[] listfiles()** devuelve un array de tipo File (File[]) con todos los elementos contenidos en la carpeta (representados por objetos File, uno por elemento). Para que se ejecute correctamente la ruta debe indicar una carpeta. El tamaño del vector será igual al número de elementos que contiene la carpeta. Si el tamaño es 0, el valor devuelto será null y toda operación posterior sobre el vector será errónea. El orden de los elementos es aleatorio (al contrario que en el explorador de archivos del sistema operativo, no se ordena automáticamente por tipo ni alfabéticamente).

```
public static void main(String[] args) {  
    // TODO Auto-generated method stub  
    File directorio= new File  
    ("/Users/jakelinsayel/Desktop/Documentos_Ficheros/Libros");  
    File[] lista=directorio.listFiles();  
  
    //Recorremos Array  
  
    for (int i=0;i<lista.length; i++) {  
        File f=lista[i];  
        if(f.isDirectory()) {  
            System.out.println("[DIR] " + f.getName());  
        }else {  
            System.out.println("[ARCH] " + f.getName());  
        }  
    }  
}
```


6. ACCESO A FICHEROS CON FLUJOS DE DE BYTES

Streams de Bytes

- Son necesarios para poder manejar cualquier tipo de fichero externo
- Cualquier tipo de archivo que tengamos dentro de un ordenador está formado por una sucesión bytes (valor entre 0-255)
- Un fichero, es un montón de bytes que todos juntos forman dicho fichero ya sea de texto, audio, video...etc.

```
class Leer_Fichero{  
    public void lee(){  
        try {  
            entrada=new FileReader("C:/Users/Juan/Desktop/ejemplo.txt");  
            int c=0;  
            while(c!=-1){  
                c=entrada.read();  
                char letra=(char)c;  
                System.out.print(letra);  
            }  
            //entrada.close();  
        } catch (IOException e) {  
            // TODO Auto-generated catch block  
            System.out.println("No se ha encontrado el archivo");  
        }finally{  
            try {  
                entrada.close();  
            } catch (IOException e) {  
                // TODO Auto-generated catch block  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

Stream byte: Enteros entre 0 y 255

65 125 45 225 185 195 215 45 10 250

Sucesión de bytes que
forman el fichero



6. ACCESO A FICHEROS CON FLUJOS DE DE BYTES

Streams de Bytes

- Si queremos manejar un fichero externo desde un programa de Java, lo primero que tendremos que hacer es descomponer esa sucesión de bytes.
- Después podemos manipularlos como necesitemos, bien desde el fichero al programa de java o viceversa (lectura, escritura). Todo irá a través de un Stream de bytes.
- Para ello usaremos dos clases:



FileInputStream

Lectura de sucesión de bytes

read()

close()

Consultar
API

FileOutputStream

Escritura de sucesión de bytes

write()

close()

6.ACCESO A FICHEROS CON FLUJOS DE DE BYTES

Streams de Bytes: LECTURA

```
public class EjemploBinario01 {
    public static void main(String[] args) {
        try {
            FileInputStream LeerArchiv = new FileInputStream("/Users/jakelinsayel/Desktop/Documentos_Ficheros/Fotografias/astronauta.jpg");
            boolean finalArchiv=false; // para saber si hemos llegado al final de la lectura
            while(!finalArchiv) {
                int bytelImagenEntrada=LeerArchiv.read();
                //para salir del bucle comprobamos si hemos llegado al final
                if(bytelImagenEntrada==-1) finalArchiv=true;
                System.out.println(bytelImagenEntrada); // solo para ver la sucesión de bytes
            }
            LeerArchiv.close();
        } catch (IOException e) { //ojo!! read() lanza esta excepción que está por encima de la de leer fichero

            // TODO Auto-generated catch block

            e.printStackTrace();
        }
    }
}

/** El método read(), en este caso va leyendo byte a byte, es decir, descompone la imagen y lee cada uno de los bytes que la forman
 * devuelve -1 cuando termina
 */
```

6.ACCESO A FICHEROS CON FLUJOS DE DE BYTES

Streams de Bytes: Escritura

```
public static void main(String[] args) {  
    // Ruta de la imagen a leer  
    String rutaImagen = "/Users/jakelinDesktop/Documentos_Ficheros/Fotografias/astronauta.jpg";  
    // Ruta del archivo de destino  
    String rutaNuevoArchivo = "/Users/jakelinDesktop/Documentos_Ficheros/Fotografias/FicheroExterniCreado/NuevoAstronauta.jpg";  
    // Leer la imagen y almacenarla en bytesImagen  
    List<Integer> bytesImagen = leerImagen(rutaImagen);  
    if (bytesImagen != null) {  
        // Escribir la imagen en el archivo de destino  
        CrearFichero(bytesImagen, rutaNuevoArchivo);  
    } else {  
        System.out.println("Error al leer la imagen.");  
    }  
}
```

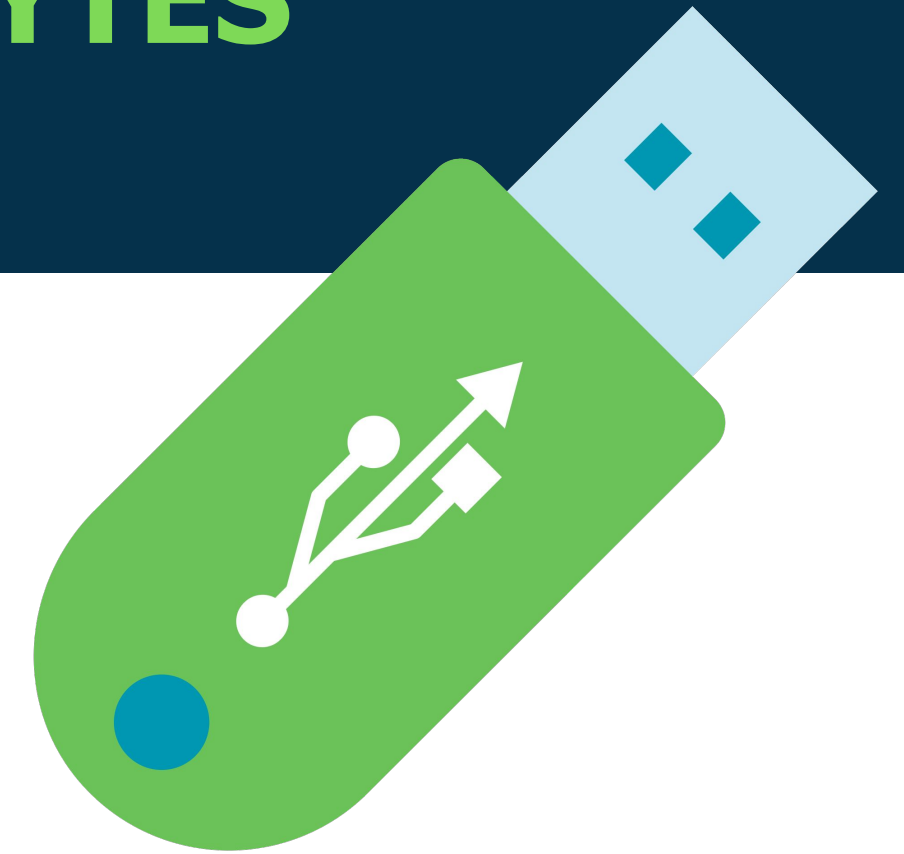


6.ACCESO A FICHEROS CON FLUJOS DE DE BYTES

// Método para leer la imagen desde un archivo

```
static List<Integer> leerImagen(String rutaImagen) {
    List<Integer> bytesImagen = new ArrayList<>();
    try (FileInputStream LeerArchiv = new FileInputStream(rutaImagen)) {
        int byteImagenEntrada;
        while ((byteImagenEntrada = LeerArchiv.read()) != -1) {
            bytesImagen.add(byteImagenEntrada);
        }
    } catch (IOException e) {
        System.out.println("Error al leer la imagen: " + e.getMessage());
        return null;
    }
    return bytesImagen;
}

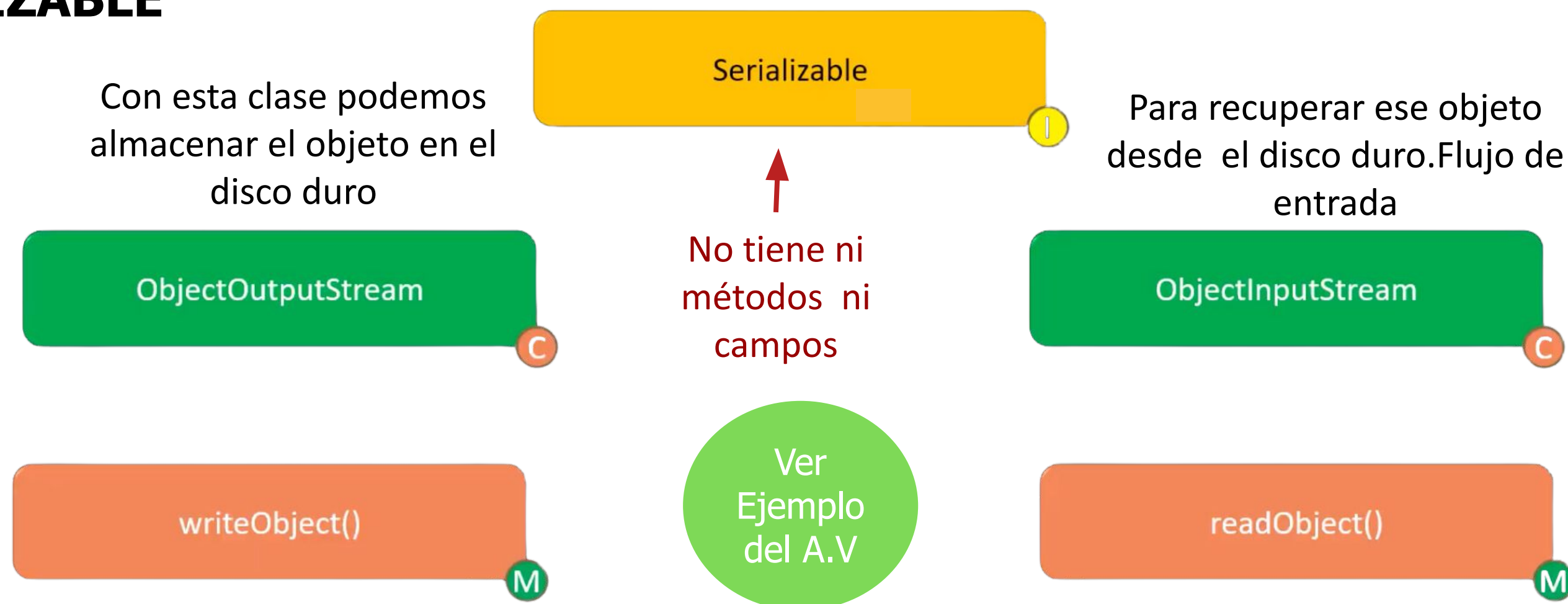
// Método para escribir la imagen en una carpeta
static void CrearFichero(List<Integer> arrayExterno, String rutaNuevoArchivo) {
    try (FileOutputStream fichExterno = new FileOutputStream(rutaNuevoArchivo)) {
        // Escribe los bytes del ArrayList en el archivo
        for (int byteExterno : arrayExterno) {
            fichExterno.write(byteExterno);
        }
        System.out.println("Imagen escrita con éxito en el archivo.");
    } catch (IOException e) {
        System.out.println("Error al escribir la imagen: " + e.getMessage());
    }
}
```



7.SERIALIZACIÓN

Ver La
API de
Java

- La serialización consiste en un objeto que tengamos en java convertirlo a bytes (empaquetarlo)
- Con el objetivo de distribuirlo donde quieras, por ejemplo en una red LAN, a través de internet...etc (sockets)
- Creamos un fichero nuevo que tiene dentro un objeto y luego lo distribuimos, para ello vamos a necesitar la interfaz **SERIALIZABLE**
- Todo objeto que queramos convertir en bytes, es decir, que queramos serializar. Sí o sí debe implementar la interfaz serializable.



7.SERIALIZACIÓN

- La serialización consiste en un objeto que tengamos en java convertirlo a bytes (empaquetarlo)
- Con el objetivo de distribuirlo donde quieras, por ejemplo en una red LAN, a través de internet...etc



```
public static void main(String[] args) {
    Administrador jefe=new Administrador ("Lola",2300,2022,12,5);
    jefe.setIncentivo(500);
    Empleado[] person1= new Empleado[3];
    person1[0]=jefe; //polimorfismo
    person1[1]= new Empleado("Pepe",1500,2020,11,11);
    person1[2]= new Empleado("Ana",1500,2021,10,7);
    //Vamos a serializar el array
    try {//le vamos a pasar la ruta donde lo quiero guardar
        ObjectOutputStream flujoSalida= new ObjectOutputStream(new
FileOutputStream
("/Users/jakelin/Desktop/Documentos_Ficheros/ArraySerializado/objSerializado.dat"
));
        flujoSalida.writeObject(person1);//para poder escribirlo
        flujoSalida.close();
        //OJO!! REcordad que Empleados debe implementar SERIALIZABLE
    } catch (FileNotFoundException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    } catch (IOException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}

class Empleado implements Serializable{
    public Empleado(String n, double s, int agno, int mes, int dia){
```

7.SERIALIZACIÓN

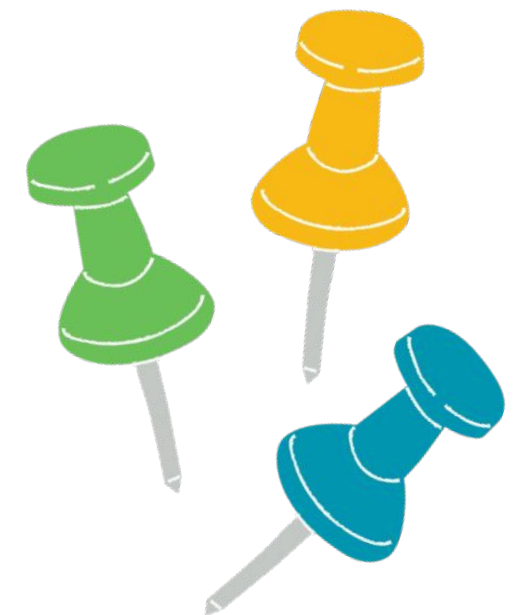
→ Recuperar la información

```
import java.io.*;
public class SerializarREcuperar02 {
    public static void main(String[] args) {
        // TODO Auto-generated method stub

        try {
            ObjectInputStream flujoEntrada= new ObjectInputStream (new
FileInputStream("/Users/jakelinsayel/Desktop/Documentos_Ficheros/ArraySerializado/objSerializado.dat
"));
            //como entra una variable de tipo array necesito otra varibale de tipo array para guardarla
            Empleado[] entradaObj= (Empleado[]) flujoEntrada.readObject();
            flujoEntrada.close();
            //Ahora vamos a comprobar si ha recogido los datos
            //debe de tener los datos de Empleados

            for(Empleado e:entradaObj) {
                System.out.println(e);
            }
            //imprime en consola la información
            //del objeto serializado

        } catch (Exception e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
    }
}
```





¿Preguntas?